

INTERBLOQUEOS

DEPARTAMENTO DE CIENCIAS E INGENIERÍA
DE LA COMPUTACIÓN

UNIVERSIDAD NACIONAL DEL SUR

KMC © 2026

Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

Agenda

1. **Problema**
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

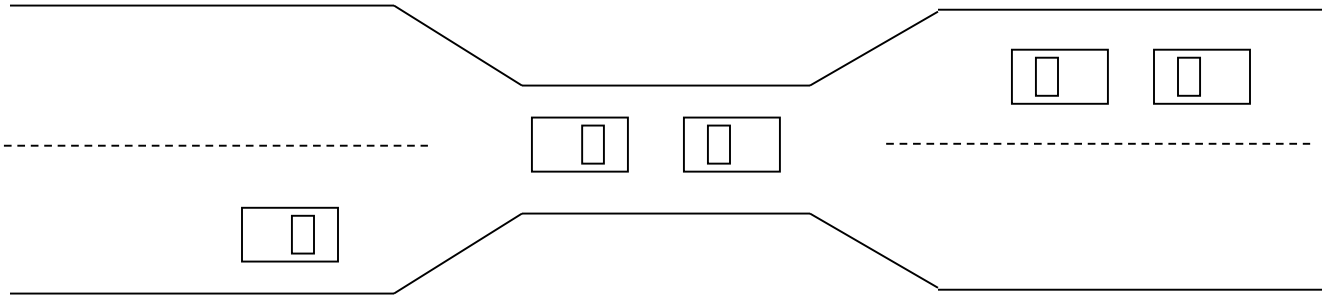
El Problema de Interbloqueo

Un conjunto de procesos están bloqueados, cuando cada uno contiene recursos y espera para adquirir otro recurso mantenido por otro proceso en el conjunto.

- Ejemplo 1
  - Un sistema tiene 2 discos.
  - P_1 y P_2 cada uno tiene un disco y necesita otro.
- Ejemplo 2
  - semáforos *A* y *B*, inicializados a 1

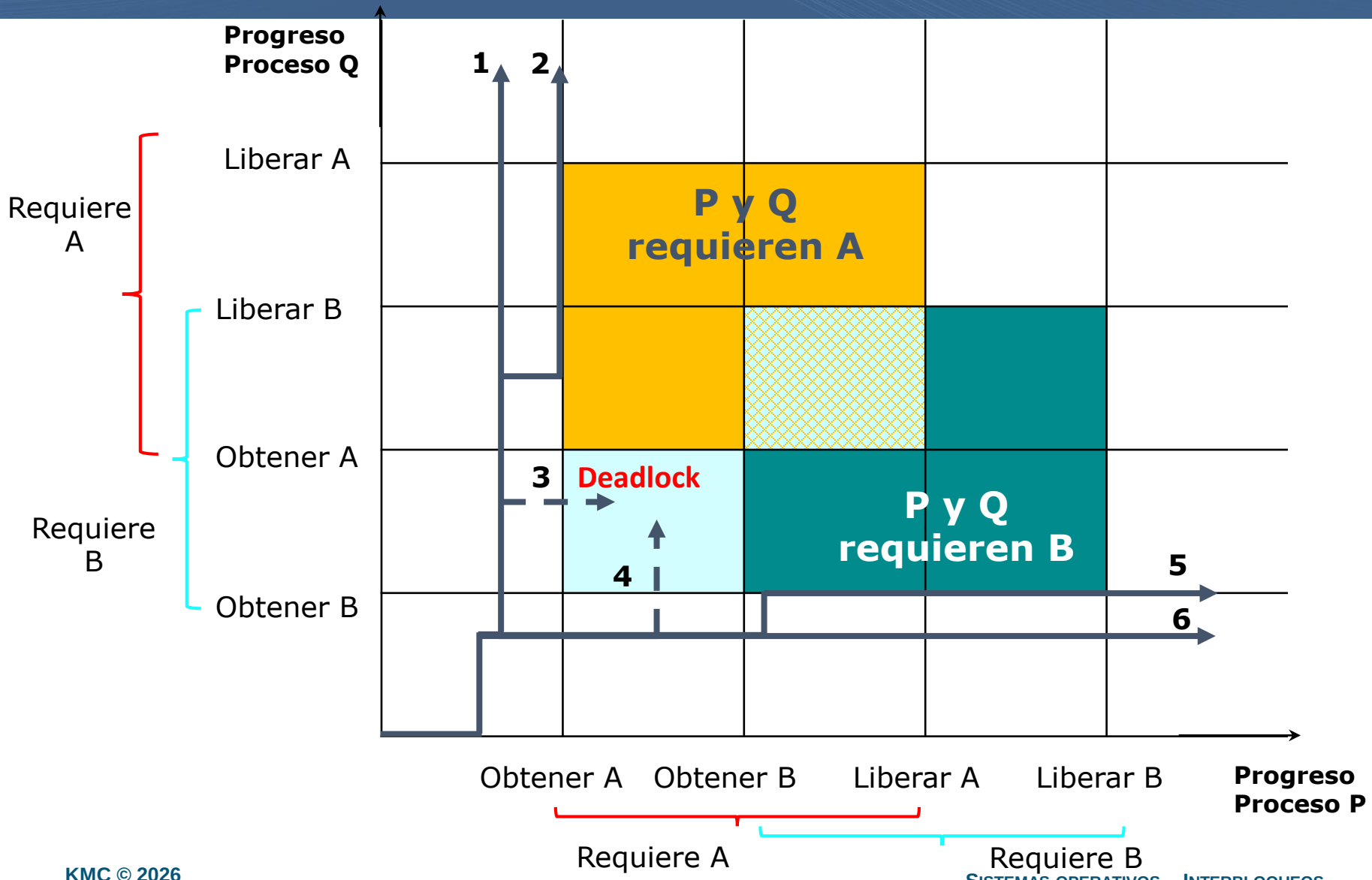
P_0	P_1
wait (A);	*wait(B)*
wait (B);	*wait(A)*

Ejemplo del Cruce en el Puente



- Tránsito en una sola dirección.
- Cada sección de un puente puede ser vista como un recurso.
- Si un interbloqueo ocurre, puede ser resuelto si cada auto retrocede (se apropia de recursos y *rollback*).
- Varios autos pueden retroceder si un interbloqueo occurre.
- Es posible la inanición.

Ejemplo de Interbloqueos



Agenda

1. Problema
2. **Modelo de Sistema**
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

Modelo de Sistema

- Tipos de Recursos $R_1, R_2, \ldots, R_m$

  ciclos CPU, espacio de memoria, dispositivos E/S

- Cada recurso tipo R_i tiene W_i instancias.
- Cada proceso utiliza un recurso como sigue:
  - **requerimiento**
  - **uso**
  - **liberalización**

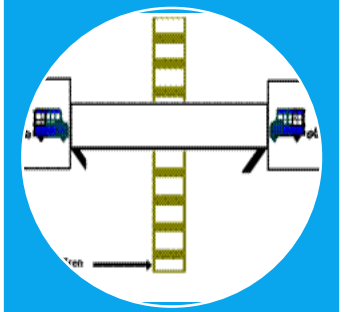


Agenda

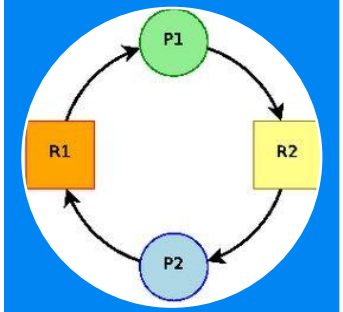
1. Problema
2. Modelo de Sistema
3. **Caracterización de Interbloqueos**
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

Caracterización de Interbloqueos

El Interbloqueo puede alcanzarse si se cumplen las cuatro condiciones simultáneamente.



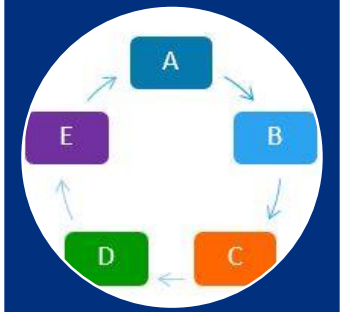
Exclusión Mutua



Retener y Esperar



No apropiación



Espera circular

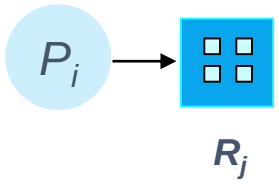
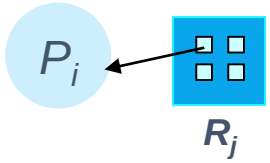
KMC © 2026

Grafo de Alocación de Recursos

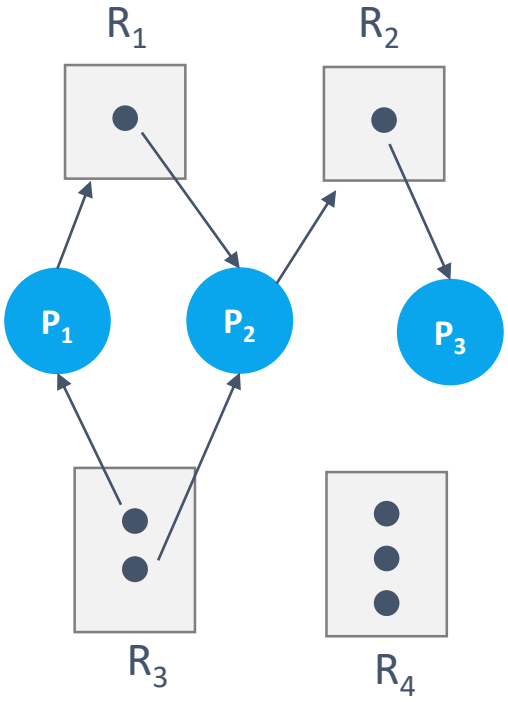
Un conjunto de vértices V y un conjunto de lados E.

- V está particionado en dos tipos:
  - $P = \{P_1, P_2, \ldots, P_n\}$, el conjunto consistente de todos los procesos en el sistema.
  - $R = \{R_1, R_2, \ldots, R_m\}$, el conjunto consistente de todos los recursos tipo en el sistema.
- lado de requerimiento – lado dirigido $P_i \rightarrow R_j$
- lado de asignamiento – lado dirigido $R_j \rightarrow P_i$

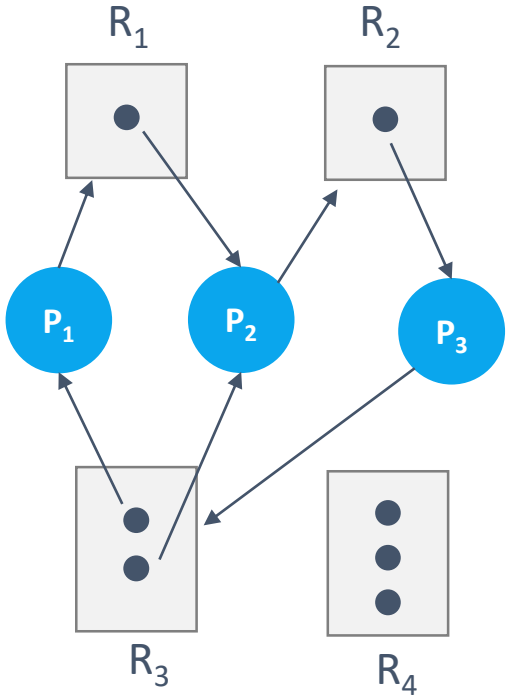
Grafo de Alocación de Recursos

Elemento	Representación
Proceso	
Recurso	
lado dirigido $P_i \rightarrow R_j$	
lado dirigido $R_j \rightarrow P_i$	

Ejemplos: Grafo de Alocación de Recursos

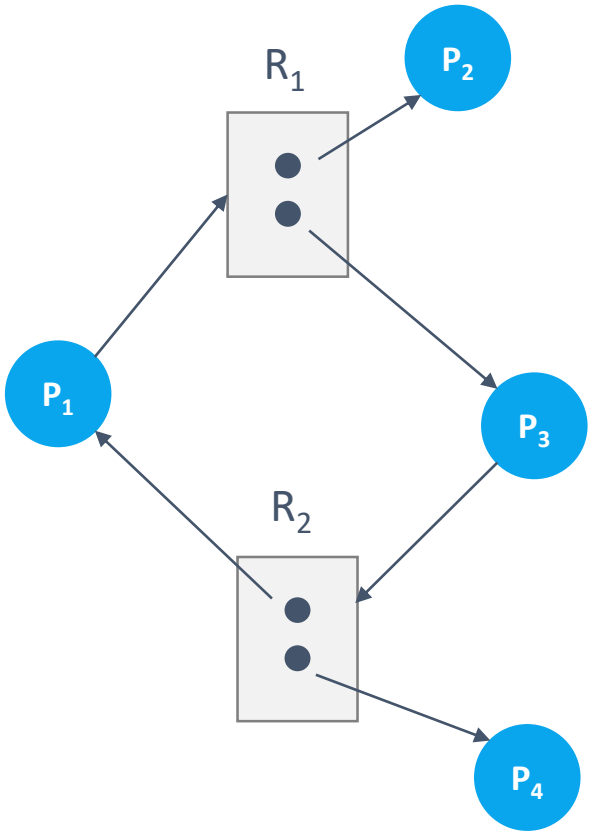


a) Grafo sin ciclo



b) Grafo con interbloqueo

Ejemplos: Grafo de Alocación de Recursos



c) Grafo con ciclo sin interbloqueo

Cuestiones Básicas

- Si un grafo no contiene ciclos $\Rightarrow$ no hay interbloqueo.
- Si un grafo contiene un ciclo $\Rightarrow$
  - Si hay una sola instancia por tipo de recurso, entonces hay interbloqueo.
  - Si hay varias instancias por tipo de recurso, hay posibilidad de interbloqueo.

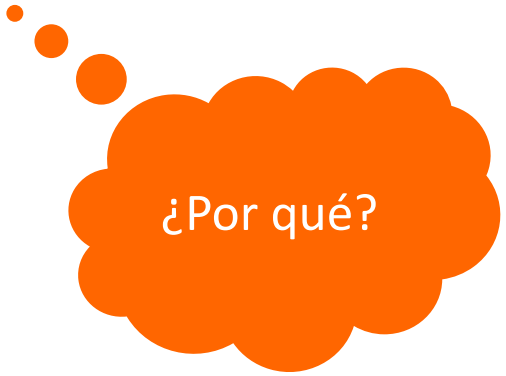
Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. **Métodos para el Manejo de Interbloqueos**
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

KMC © 2026

Métodos para Manejo de Interbloqueos

- Asegure que el sistema no entrará *nunca* estado de interbloqueo.
- Permitir al sistema entrar en un estado de interbloqueo y luego recuperarse.
- Ignore el problema y pretenda que el interbloqueo nunca ocurrió en el sistema; usado en la mayoría de los sistemas operativos incluído UNIX, Linux, Windows.



Estrategias para Manejo de Interbloqueos

Las estrategias utilizadas para atacar el problema de interbloqueos se establecen en tres niveles de acción: antes que suceda, cuando los procesos están corriendo o con hechos consumados.

Estas estrategias son:



Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. **Prevención de Interbloqueos**
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

Prevención de Interbloqueos

Restringir los modos en que se pueden hacer los requerimientos

- **Exclusión Mutua** – no requerido para recursos compartidos; debe mantenerse para recursos no compartidos.
- **Mantener y Esperar** – debe garantizar que siempre que un proceso requiera un recurso no mantiene otros.
- **No Apropiación** - Si un proceso que mantiene algunos recursos requiere otro recurso, no le puede ser inmediatamente alocado, entonces todos los recursos mantenidos son liberados.
- **Espera Circular** – impone un orden total de todos los tipos de recursos, y requiere que cada proceso requiera recursos en un orden creciente de enumeración.

Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. **Evasión de Interbloqueos**
7. Detección de Interbloqueos
8. Recuperación de Interbloqueos

KMC © 2026

Evasión de Interbloqueos

Requiere que el sistema tenga disponible alguna información adicional *a priori*.

- El modelo más simple y útil requiere que cada proceso declare el *máximo número* de recursos de cada tipo que puede necesitar.
- El algoritmo de evasión de interbloqueos dinámicamente examina el estado de alocación de recursos para asegurar que no puede haber una condición de espera circular.
- El *estado* de alocación de recursos está definido por el número de recursos disponibles y alocados, y las máximas demandas de los procesos.

Estado Seguro

- Cuando un proceso requiere un recurso disponible, el algoritmo debe decidir si su alocación inmediata deja al sistema en un estado seguro.
- El sistema está en un **estado seguro** si existe una secuencia segura de ejecución de todos los procesos.
- La secuencia $<P_1, P_2, \ldots, P_n>$ es segura si por cada P_i, los recursos que P_i puede aún requerir pueden ser satisfechos por recursos disponibles corrientes más los recursos mantenidos por todos los P_j, con $j<i$.
  - Si los recursos que P_i necesita no están inmediatamente disponibles, entonces P_i puede esperar hasta que todos los P_j hayan finalizado.
  - Cuando P_j finaliza, P_i obtiene los recursos necesitados, ejecuta, retorna los recursos alocados, y termina.
  - Cuando P_i termina, P_{i+1} puede obtener los recursos que necesita, y así sucesivamente.

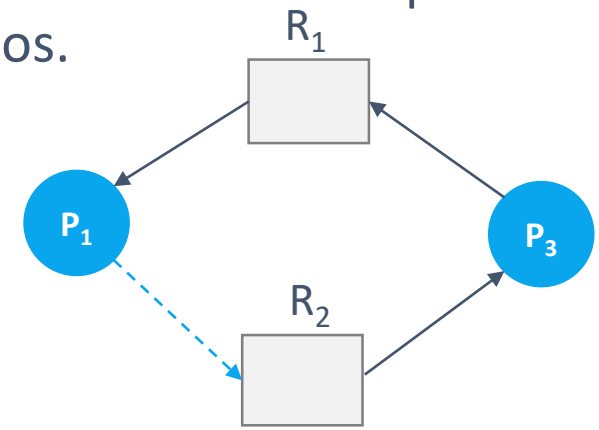
Cuestiones Básicas

- Si un sistema está en un estado seguro ⇒ no hay interbloqueo.
- Si un sistema está en un estado inseguro ⇒ posibilidad de interbloqueo.
- Evasión ⇒ asegura que el sistema *nunca* va a entrar en un estado inseguro.



Algoritmos de Evasión

- Si sólo hay una instancia de un tipo de recurso. Use un grafo de alocación de recursos.



- Para múltiple instancias de un tipo de recursos. Use el algoritmo del banquero.

Algoritmos de Evasión: Ejemplo

Considerar 12 recursos iguales y 3 procesos con las siguientes necesidades máximas y alocación corriente.

	Máxima	Alocados
P_0	10	5
P_1	4	2
P_2	9	2

Sec. segura <P_1,P_0,P_2>

probar P_2 con alocación=3

Sec. insegura <P_1,……..>

Algoritmo del Banquero

- Múltiples instancias.
- Cada proceso debe reclamar a priori el máximo uso.
- Cuando un proceso requiere un recurso puede tener que esperar.
- Cuando un proceso obtiene todos sus recursos debe retornarlos en una cantidad finita de tiempo.

Algoritmo del Banquero: Estructuras de Datos

Sea *n* = número de procesos, y *m* = número de tipos de recursos.

- *Disponible:* Vector de longitud *m*. Si disponible[*j*] = *k*, hay *k* instancias del tipo de recurso R_j disponible.
- *Max:* matriz *n x m*. Si *Max*[*i,j*] = *k*, entonces el proceso P_i puede requerir a lo sumo *k* instancias del recurso de tipo R_j.
- *Alocación:* matriz *n x m*. Si Alocación[*i,j*] = *k* entonces P_i tiene alocadas *k* instancias de R_j.
- *Necesidad:* matriz *n x m*. Si *Necesidad*[*i,j*] = *k*, entonces P_i puede necesitar *k* instancias más de R_j para completar su tarea.

Necesidad[*i,j*] = *Max*[*i,j*] – *Alocación*[*i,j*].

Algoritmo de Seguridad

1. Sean *Trab y Final* vectores de longitud *m* y *n*, respectivamente. Se inicializan:

   Trab := *Disponible*

   Final[*i*] = *falso para i* - 1,2, …, *n.*

2. Encuentre un *i* tal que cumplan:

   (a) *Final*[*i*] = *falso*

   (b) $Necesidad_i \leq Trab$

   Si tal *i no* existe, vaya al paso 4.

3. *Trab* := *Trab* + $Alocación_i$
   Final[*i*] := *verdadero*
   vaya al paso 2.

4. Si *Final*[*i*] = verdadero para todo *i*, entonces el sistema está en un estado seguro.

Algoritmo de Requerimiento de Recursos para el Proceso P_i

$Request_i$ = vector de requerimento para el proceso P_i.
Si $Request_i[j] = k$ entonces el proceso P_i quiere k instancias del tipo de recurso R_j.

1. Si $Request_i \leq Necesidad_i$ vaya al paso 2. Sino **condición de error**, dado que el proceso ha excedido su máximo.
2. Si $Request_i \leq Disponible$, vaya al paso 3. Sino P_i debe esperar, dado que los recursos no están disponibles.
3. Se simula alocar los recursos requeridos P_i modificando el estado como sigue:

   $Disponible := Disponible - Request_i;$

   $Alocación_i := Alocación_i + Request_i;$

   $Necesidad_i := Necesidad_i - Request_i;$

   - *Si **seguro** $\Rightarrow$ los recursos son alocados a P_i.*
   - *Si **inseguro** $\Rightarrow$ P_i debe esperar, y es restaurado el viejo estado de alocación de recursos*

Algoritmo del Banquero: Ejemplo

- 5 procesos P_0 a P_4; 3 recursos tipo *A* (10 instancias), *B* (5 instancias), y *C* (7 instancias).
- Instantánea en el instante T_0:

	Alocación	*Max*	*Disponible*	***Necesidad***
	A B C	*A B C*	*A B C*	***A B C***
P_0	0 1 0	7 5 3	3 3 2	**7 4 3**
P_1	2 0 0	3 2 2		**1 2 2**
P_2	3 0 2	9 0 2		**6 0 0**
P_3	2 1 1	2 2 2		**0 1 1**
P_4	0 0 2	4 3 3		**4 3 1**

El sistema está en un estado seguro dado que la secuencia $< P_1, P_3, P_4, P_2, P_0 >$ satisface los criterios de seguridad.

Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. **Detección de Interbloqueos**
8. Recuperación de Interbloqueos

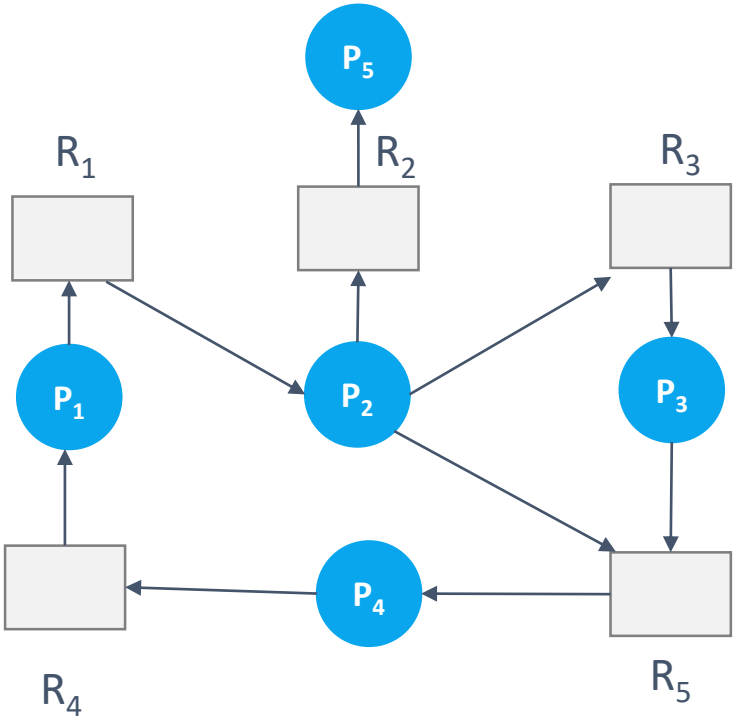
Detección de Interbloqueos

- Permite al sistema entrar en estado de interbloqueo
- Algoritmo de Detección
- Esquema de Recuperación

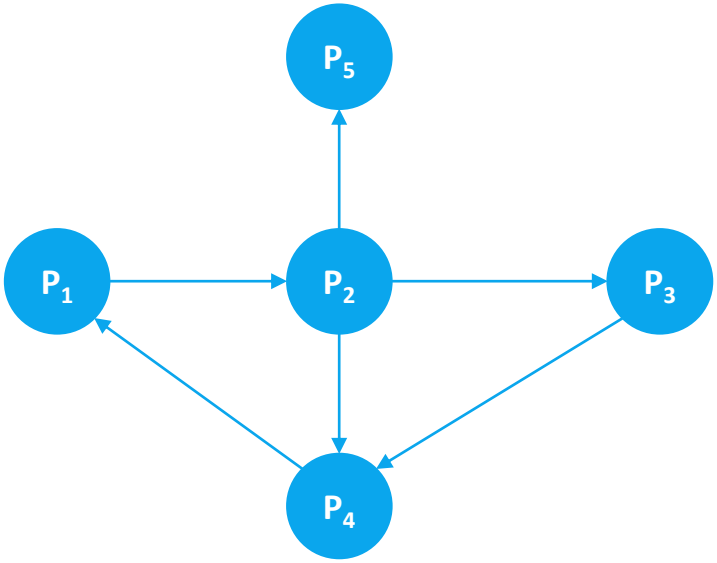
Instancia Simple de Cada Tipo de Recurso

- Se mantiene un grafo *wait-for*
  - Los nodos son procesos.
  - $P_i \rightarrow P_j$ si P_i está esperando por P_j.
- Periódicamente se invoca un algoritmo que busca por ciclos en el grafo.
- Un algoritmo para detectar un ciclo en un grafo requiere un orden de n^2 operaciones, donde n es el número de vértices en el grafo.

Grafo de Alocación de Recursos y Grafo Wait-for



Grafo de alocación de recursos



Grafo wait-for correspondiente

Varias Instancias de un Tipo de Recurso

- *Disponible:* Vector de longitud *m* indica el número de recursos disponibles de cada tipo.
- *Alocación:* Una matriz de *n x m* que define el número de recursos de cada tipo corrientemente alocados por cada proceso.
- *Request:* Una matriz de *n x m* que indica el requerimiento corriente de cada proceso. Si *Request*[*i,j*] = *k*, entonces el proceso P_i está requiriendo *k* instancias más del recurso de tipo R_j.

Algoritmo de Detección

1. Sean *Trab* y *Final* vectores de longitud *m* y *n*, respectivamente inicializados:
   (a) *Trab* = *Disponible*
   (b) Para *i* = 1,2, …, *n*, si $Alocación_i \neq 0$, entonces
   Final[i] := falso; sino *Final*[i] := verdadero.
2. Encuentre un índice *i* tal que:
   (a) *Final*[*i*] = *falso*
   (b) $Request_i \leq Trab$
   Si no existe tal *i*, vaya al paso 4.
3. *Trab* := *Trab* + $Alocación_i$
   Final[*i*] := *verdadero*
   vaya al paso 2.
4. Si *Final*[*i*] = falso, para algún *i*, $1 \leq i \leq n$, entonces el sistema está en un estado de interbloqueo. Es mas, si *Final*[*i*] = *falso*, entonces P_i está interbloqueado.

El algoritmo requiere un orden de O(m x $n^{2)}$ operaciones para detectar si el sistema está en un estado de interbloqueo.

Algoritmo de Detección: Ejemplo

- Cinco procesos P_0 a P_4; tres tipos de recursos A (7 instancias), *B* (2 instancias), y *C* (6 instancias).
- En el instante T_0:

	Alocación	*Request*	*Disponible*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- La secuencia <P_0, P_2, P_3, P_1, P_4> resultará en *Final*[*i*] = verdadero para todo *i*.

Uso del Algoritmo de Detección

- Cuándo y qué frecuente debe invocarse depende de:
  - ¿Con qué ***frecuencia*** es probable que ocurra un interbloqueo?
  - ¿***Cuántos*** procesos serán afectados cuando ocurra un interbloqueo?
- Si el algoritmo de detección es invocado arbitrariamente, puede haber muchos ciclos en el grafo de recursos y no se puede decir cual de los muchos procesos interbloqueados **causó** el interbloqueo.

Agenda

1. Problema
2. Modelo de Sistema
3. Caracterización de Interbloqueos
4. Métodos para el Manejo de Interbloqueos
5. Prevención de Interbloqueos
6. Evasión de Interbloqueos
7. Detección de Interbloqueos
8. **Recuperación de Interbloqueos**

Recuperación de Interbloqueos: Terminación de Procesos

- Aborto de todos los procesos interbloqueados.
- Aborto de un proceso a la vez hasta que el ciclo de interbloqueo haya sido eliminado.
- ¿En qué orden se elige abortar?
  - Prioridad del proceso.
  - Cuánto tiempo ha computado el proceso, y cuánto falta para completar.
  - Recursos que el proceso ha usado.
  - Recursos que el proceso necesita para completar.
  - Cuántos procesos necesitarán ser terminados.
  - ¿Es el proceso interactivo o batch?

Recuperación de Interbloqueos: Apropiación de Recursos

- Selección de una víctima – minimiza costos.
- Rollback – retorna a algún estado seguro, reinicia el proceso desde ese estado.
- Inanición – algunos procesos pueden ser elegidos siempre como víctimas, incluir un número de rollbacks en el factor de costo.

Bibliografía:

- Silberschatz, A., Gagne G., y Galvin, P.B.; "Operating System Concepts", 7ma Edición 2009, 9na Edición 2012, 10ma Edición 2018.
- Stallings, W. "Operating Systems: Internals and Design Principles", Prentice Hall, 6ta Edición 2009, 7ma Edición 2011, 9na Edición 2018.